

# Detailed Notes on OOP Lecture 6: Packages, Access Modifiers, Encapsulation, Static and Final Keywords, Pass by Value vs. Reference

This document provides comprehensive notes for the sixth video of the Community Classroom's Object-Oriented Programming (OOP) course in Java, focusing on packages, access modifiers, encapsulation, static and final keywords, and pass-by-value vs. pass-by-reference. The notes are organized by the topics covered in the video, including key concepts, code snippets, and examples, with additional explanations to ensure clarity and retention for aspiring elite coders.

## 1 Packages in Java

### 1.1 Key Concepts

- **Package:** A namespace that organizes related classes and interfaces, preventing naming conflicts and providing access control.
- **Purpose:** Enhances code organization, modularity, and maintainability; protects classes from unauthorized access.
- **Declaring a Package:** Use the `package` keyword at the top of a Java file.
- **Importing Packages:** Use `import` to access classes from other packages.
- **Common Packages:**
  - `java.lang`: Automatically imported (e.g., `String`, `System`).
  - `java.util`: Utility classes (e.g., `ArrayList`, `Scanner`).
  - `java.io`: Input/output operations.
- **Naming Convention:** Typically lowercase, reverse domain (e.g., `com.example.myapp`).

### 1.2 Explanation

The video explains that packages group related classes, similar to folders in a file system. They prevent naming conflicts (e.g., two classes named `Student` in different packages) and allow access control via modifiers. The video emphasizes using packages to structure large projects.

### 1.3 Example from Video

The video demonstrates creating a package `com.company` with a `Employee` class and accessing it from another package.

### 1.4 Code Snippet

```
1 // File: com/company/Employee.java
2 package com.company;
3
4 public class Employee {
5     String name = "John Doe";
6     void display() {
7         System.out.println("Employee: " + name);
8     }
9 }
10
11 // File: Main.java
12 import com.company.Employee;
13
14 public class Main {
15     public static void main(String[] args) {
16         Employee emp = new Employee();
17         emp.display(); // Output: Employee: John Doe
18     }
19 }
```

#### Important Points:

- Packages must match the directory structure (e.g., `com.company.Employee` resides in `com/company/Employee.java`).
- Use `import` to access classes across packages or fully qualified names (e.g., `com.company.Employee`).
- Packages improve scalability in large projects, a must-know for enterprise-level coding.

## 2 Access Modifiers

### 2.1 Key Concepts

- **Access Modifiers:** Control the visibility and accessibility of class members (fields, methods, constructors).
- **Types:**
  - `public`: Accessible from everywhere.
  - `protected`: Accessible within the same package and in subclasses (even in different packages).
  - `default` (package-private): Accessible only within the same package if no modifier is specified.

- **private:** Accessible only within the same class.
- **Purpose:** Enforce encapsulation, protect data, and control access to class members.

## 2.2 Explanation

The video highlights that access modifiers are critical for encapsulation, ensuring that sensitive data or methods are only accessible where needed. It explains the scope of each modifier with practical examples.

## 2.3 Example from Video

The video uses a `BankAccount` class to demonstrate how access modifiers control access to fields like `balance`.

## 2.4 Code Snippet

```

1 package bank;
2
3 public class BankAccount {
4     private double balance = 1000.0; // Only accessible within
        this class
5     protected String accountHolder = "Alice"; // Accessible in
        package and subclasses
6     public void deposit(double amount) { // Accessible everywhere
7         balance += amount;
8     }
9     void display() { // Package-private (default)
10         System.out.println("Balance: " + balance);
11     }
12 }
13
14 // File: Test.java (same package)
15 package bank;
16
17 public class Test {
18     public static void main(String[] args) {
19         BankAccount account = new BankAccount();
20         account.deposit(500.0);
21         account.display(); // Output: Balance: 1500.0
22         // System.out.println(account.balance); // Error: balance
            is private
23         System.out.println(account.accountHolder); // Output:
            Alice
24     }
25 }
26
27 // File: SubClass.java (different package)
28 package other;
29
30 import bank.BankAccount;

```

```

31
32 public class SubClass extends BankAccount {
33     void show() {
34         System.out.println(accountHolder); // Accessible (
           protected)
35         // System.out.println(balance); // Error: balance is
           private
36     }
37 }

```

### Important Points:

- `private` fields ensure data security, critical for robust systems.
- `protected` is useful for inheritance across packages.
- Default access is restrictive, suitable for package-level encapsulation.
- Use `public` sparingly to avoid exposing unnecessary details.

## 3 Encapsulation

### 3.1 Key Concepts

- **Encapsulation:** The bundling of data (fields) and methods that operate on that data, restricting direct access to some of an objects components.
- **Implementation:** Use `private` fields with `public` getter and setter methods to control access.
- **Benefits:** Data hiding, improved maintainability, flexibility to change internal implementation without affecting external code.

### 3.2 Explanation

The video emphasizes encapsulation as a pillar of OOP, ensuring data integrity by hiding internal state and exposing only necessary interfaces. It stresses using getters and setters to validate data.

### 3.3 Example from Video

The video revisits the `BankAccount` class, adding getters and setters to safely access and modify `balance`.

### 3.4 Code Snippet

```

1 package bank;
2
3 public class BankAccount {
4     private double balance;
5     private String accountHolder;
6
7     // Constructor

```

```

8      public BankAccount(String accountHolder, double
9          initialBalance) {
10         this.accountHolder = accountHolder;
11         this.balance = initialBalance;
12     }
13
14     // Getter
15     public double getBalance() {
16         return balance;
17     }
18
19     // Setter with validation
20     public void setBalance(double balance) {
21         if (balance >= 0) {
22             this.balance = balance;
23         } else {
24             System.out.println("Invalid balance!");
25         }
26     }
27
28     public String getAccountHolder() {
29         return accountHolder;
30     }
31
32     public void setAccountHolder(String accountHolder) {
33         this.accountHolder = accountHolder;
34     }
35 }
36
37 package bank;
38
39 public class Main {
40     public static void main(String[] args) {
41         BankAccount account = new BankAccount("Alice", 1000.0);
42         account.setBalance(1500.0);
43         System.out.println("Balance: " + account.getBalance());
44         // Output: Balance: 1500.0
45         account.setBalance(-100.0); // Output: Invalid balance!
46         System.out.println("Account Holder: " + account.
47             getAccountHolder()); // Output: Account Holder: Alice
48     }
49 }

```

### Important Points:

- Encapsulation prevents invalid data (e.g., negative balance) through setter validation.
- Getters provide read-only access to private fields, maintaining data integrity.
- Encapsulation is a cornerstone of secure, maintainable code, widely used in enterprise applications.

## 4 Static Keyword

### 4.1 Key Concepts

- **Static:** Indicates that a member (field, method, or nested class) belongs to the class itself, not to instances.
- **Uses:**
  - **static fields:** Shared across all instances (e.g., a counter for objects created).
  - **static methods:** Utility methods that don't require an instance (e.g., `Math.sqrt()`).
  - **static blocks:** Initialize static fields when the class is loaded.
- **Key Rules:**
  - Static members are accessed using the class name (e.g., `ClassName.staticField`).
  - Non-static members cannot be accessed directly from static contexts.

### 4.2 Explanation

The video explains that `static` is used for shared resources or utility functions. It contrasts instance-specific (non-static) and class-level (static) members, emphasizing memory efficiency.

### 4.3 Example from Video

The video uses a `Counter` class with a static field to track the number of objects created.

### 4.4 Code Snippet

```
1 public class Counter {
2     private static int count = 0; // Shared across all instances
3     private String name;
4
5     public Counter(String name) {
6         this.name = name;
7         count++;
8     }
9
10    public static int getCount() {
11        return count;
12    }
13
14    public String getName() {
15        return name;
16    }
17 }
18
19 public class Main {
20     public static void main(String[] args) {
21         Counter c1 = new Counter("Object1");
```

```

22     Counter c2 = new Counter("Object2");
23     System.out.println("Total objects: " + Counter.getCount()
24         ); // Output: Total objects: 2
25     System.out.println("Name of c1: " + c1.getName()); //
26         Output: Name of c1: Object1
    }
}

```

### Important Points:

- **static** fields are initialized once and shared, reducing memory overhead.
- **static** methods are ideal for utility functions (e.g., `Math.random()`).
- Avoid overusing **static**, as it can break OOP principles like encapsulation.

## 5 Final Keyword

### 5.1 Key Concepts

- **Final:** Indicates that a variable, method, or class cannot be modified or extended.
- **Uses:**
  - **final** variable: Constant, cannot be reassigned.
  - **final** method: Cannot be overridden by subclasses.
  - **final** class: Cannot be extended (e.g., `String` is final).
- **Purpose:** Ensures immutability, enhances security, and prevents unintended modifications.

### 5.2 Explanation

The video stresses that **final** ensures immutability, critical for thread safety and design clarity. It explains the difference between **final** fields (constants) and **final** methods/-classes (preventing override/extension).

### 5.3 Example from Video

The video uses a `Config` class with a **final** field for a constant and a **final** class to prevent extension.

### 5.4 Code Snippet

```

1 final class Config { // Cannot be extended
2     public final int MAX_USERS = 100; // Constant
3
4     final void display() { // Cannot be overridden
5         System.out.println("Max users: " + MAX_USERS);
6     }
7 }
8

```

```

9  // class SubConfig extends Config { } // Error: Cannot extend
    final class
10
11 public class Main {
12     public static void main(String[] args) {
13         Config config = new Config();
14         config.display(); // Output: Max users: 100
15         // config.MAX_USERS = 200; // Error: Cannot modify final
            field
16     }
17 }

```

### Important Points:

- **final** variables are constants, often used for configuration values.
- **final** methods ensure critical logic remains unchanged in subclasses.
- **final** classes (e.g., **String**) prevent unintended extensions, ensuring reliability.

## 6 Pass by Value vs. Pass by Reference

### 6.1 Key Concepts

- **Pass by Value:** Java passes all arguments by value, meaning a copy of the value is passed to methods.
- **Primitive Types:** A copy of the actual value (e.g., **int**, **double**) is passed; changes inside the method don't affect the original.
- **Reference Types:** A copy of the reference (memory address) is passed; changes to the object's state affect the original object, but reassigning the reference doesn't.
- **Key Misconception:** Java does not use pass-by-reference; it's always pass-by-value, even for objects.

### 6.2 Explanation

The video clarifies that Java's pass-by-value behavior confuses many due to object references. It explains that while object state changes are reflected, reassigning the reference inside a method doesn't affect the caller's reference.

### 6.3 Example from Video

The video demonstrates pass-by-value with a primitive (**int**) and an object (**Student**).

### 6.4 Code Snippet

```

1  class Student {
2      String name;
3
4      Student(String name) {
5          this.name = name;

```

```

6      }
7  }
8
9  public class Main {
10     // Pass by value for primitive
11     public static void changePrimitive(int num) {
12         num = 100; // Changes local copy
13     }
14
15     // Pass by value for reference
16     public static void changeObject(Student s) {
17         s.name = "Changed"; // Affects original object
18         s = new Student("New"); // Doesn't affect original
19         // reference
20     }
21
22     public static void main(String[] args) {
23         // Primitive example
24         int x = 10;
25         changePrimitive(x);
26         System.out.println("x: " + x); // Output: x: 10
27
28         // Object example
29         Student student = new Student("Original");
30         changeObject(student);
31         System.out.println("Student name: " + student.name); //
32         // Output: Student name: Changed
33     }
34 }

```

### Important Points:

- Java's pass-by-value means method changes to primitive types are local.
- For objects, changes to the objects state persist, but reassigning the reference is local.
- Understanding this is crucial for debugging and writing predictable code.

## 7 Practical Tips for Applying These Concepts

### 7.1 Key Concepts

- **Packages:** Use packages to organize large projects, following reverse domain naming (e.g., `com.yourname.project`).
- **Access Modifiers and Encapsulation:** Always use `private` fields with getters/setters to enforce data integrity.
- **Static:** Use for utility methods or shared resources, but avoid overuse to maintain OOP principles.
- **Final:** Use for constants and to protect critical methods/classes from modification.

- **Pass by Value:** Be cautious with object state changes in methods, as they persist outside.
- **Real-World Application:** These concepts are foundational for frameworks like Spring (uses packages, encapsulation, annotations) and Android development.

## 7.2 Example from Video

The video suggests building a Library system with packages for books, users, and transactions, using encapsulation and static methods for utility functions.

## 7.3 Code Snippet

```

1 // File: com/library/books/Book.java
2 package com.library.books;
3
4 public class Book {
5     private String title;
6     private static int totalBooks = 0;
7
8     public Book(String title) {
9         this.title = title;
10        totalBooks++;
11    }
12
13    public String getTitle() {
14        return title;
15    }
16
17    public static int getTotalBooks() {
18        return totalBooks;
19    }
20 }
21
22 // File: com/library/Main.java
23 package com.library;
24
25 import com.library.books.Book;
26
27 public class Main {
28     public static void main(String[] args) {
29         Book book1 = new Book("Java Programming");
30         Book book2 = new Book("OOP Concepts");
31         System.out.println("Total books: " + Book.getTotalBooks());
32         // Output: Total books: 2
33         System.out.println("Book 1 title: " + book1.getTitle());
34         // Output: Book 1 title: Java Programming
35     }
36 }

```

**Important Point:** This design demonstrates modularity (packages), data hiding (encapsulation), and shared state (static), aligning with professional coding practices.

## 8 Common Pitfalls and How to Avoid Them

### 8.1 Key Concepts

- **Package Misuse:** Incorrect package naming or structure can lead to access issues. Follow directory structure and naming conventions.
- **Improper Access Modifiers:** Exposing fields as `public` breaks encapsulation. Always use `private` with getters/setters.
- **Static Overuse:** Excessive `static` members can lead to inflexible code. Use only for shared or utility purposes.
- **Final Misuse:** Declaring unnecessary `final` members restricts flexibility. Use judiciously for constants or critical methods.
- **Pass-by-Value Confusion:** Assuming pass-by-reference can lead to bugs. Always test object state changes in methods.

### 8.2 Explanation

The video warns against common mistakes, such as exposing fields directly or misunderstanding pass-by-value, which can cause unexpected behavior in production code.

### 8.3 Example from Video

The video shows a bug where a public field is modified directly, bypassing encapsulation.

### 8.4 Code Snippet (Bug Example)

```
1 public class BadExample {
2     public int value = 10; // Bad: Public field
3 }
4
5 public class Main {
6     public static void main(String[] args) {
7         BadExample obj = new BadExample();
8         obj.value = -5; // No validation, breaks integrity
9         System.out.println("Value: " + obj.value); // Output:
              Value: -5
10    }
11 }
```

#### Fix with Encapsulation:

```
1 public class GoodExample {
2     private int value = 10;
3
4     public int getValue() {
5         return value;
6     }
7
8     public void setValue(int value) {
```

```

9         if (value >= 0) {
10             this.value = value;
11         } else {
12             System.out.println("Invalid value!");
13         }
14     }
15 }

```

**Important Point:** Encapsulation prevents invalid data modifications, ensuring robust code.

## 9 Connection to Your Goal

To become a top 1% coder by 2027 and land a high-paying tech job in San Francisco, mastering these concepts is essential:

- **Packages and Encapsulation:** Critical for organizing and securing large-scale systems, common in tech giants like Google or Meta.
- **Static and Final:** Used in frameworks (e.g., Springs **static** utilities, **final** configurations) and performance-critical code.
- **Pass by Value:** Understanding this prevents bugs in complex applications, a key skill for coding interviews.
- **Action Plan:**
  - **Practice:** Build a project (e.g., a student management system) using packages, encapsulation, and static utilities.
  - **Learn Frameworks:** Study Spring or Java EE to see these concepts in action.
  - **Coding Challenges:** Solve problems on LeetCode or HackerRank, focusing on encapsulation and static methods.
  - **Portfolio:** Contribute to open-source projects on GitHub, showcasing modular, encapsulated code.

## 10 Conclusion

This lecture covers foundational Java concepts that are critical for professional development. Packages, access modifiers, encapsulation, static and final keywords, and pass-by-value vs. pass-by-reference form the backbone of robust, scalable systems. Practice these through projects, apply them in real-world scenarios, and master them to align with your goal of becoming an elite coder.

**Source:** Community Classroom OOP Lecture 6, <https://youtu.be/W145DXs8fFg>